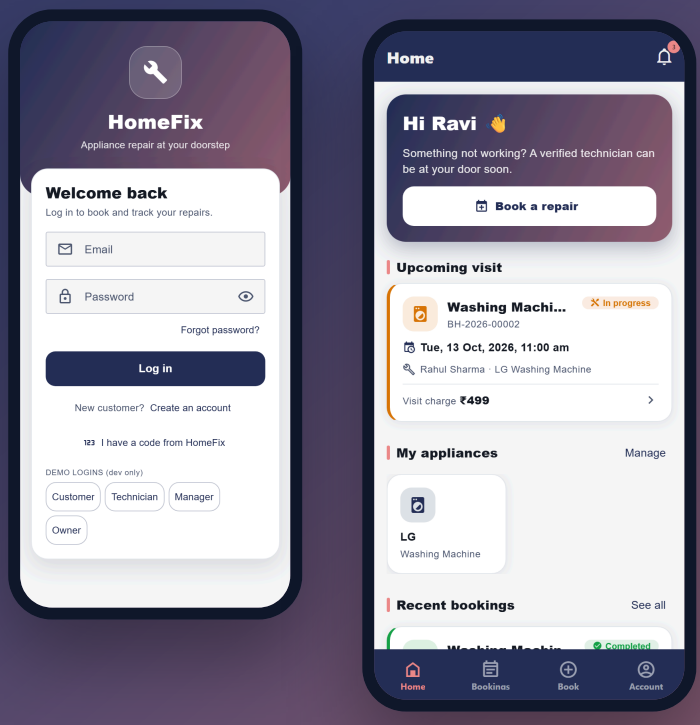
HomeFix Mobile

The complete beginner's guide: set up the project, run it, build the Android app, test Razorpay payments and record the demo.

React Native · Expo SDK 57 · TypeScript · Node.js + Express · PostgreSQL · Razorpay (test) · Playwright



Appliance repair booking & field-service app

HomeFix Appliance Services, Hyderabad

Repositories: github.com/saierukala/businesshub-reactnative · businesshub-backend

Version 1.0.0 (build 1) · 15 October 2026

INSIDE THIS GUIDE

Contents

Read it top to bottom the first time. Every command is written for a Windows PC with PowerShell; nothing is assumed.

1 What HomeFix is and how the pieces fit
2 What you need before you start
3 Get the code
4 Start the backend (database + API)
5 Set up the mobile app
6 Run it: browser, Expo Go, QR codes
7 Create your Expo account
8 Build the Android app (APK) in the cloud
9 Install the APK on your phone
10 Open the app and connect it to your PC
11 Test online payments with Razorpay
12 Record the phone screen for the demo
13 The project's files: what goes where
14 How one screen works, end to end
15 Important things to know
16 Tests, demo videos and screenshots
17 Command cheat sheet
18 Troubleshooting
19 Glossary
20 Demo logins

How to read the boxes

Blue = a helpful tip. **Amber** = be careful. **Green** = how to check it worked. **Red** = never do this. Dark blocks are commands: type them exactly, one line at a time, then press Enter.

The short version (once everything is installed)

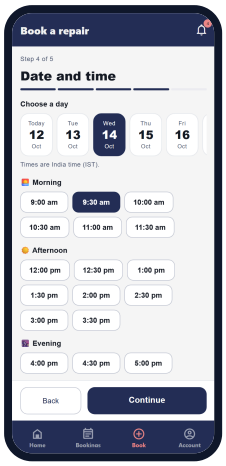
```
# Terminal 1 — the backend (API on port 4000)
cd D:\Projects\BusinessHub-backend
npm run dev

# Terminal 2 — the mobile app's code server (Metro on port 8081)
cd D:\Projects\businesshub-reactnative
npx expo start

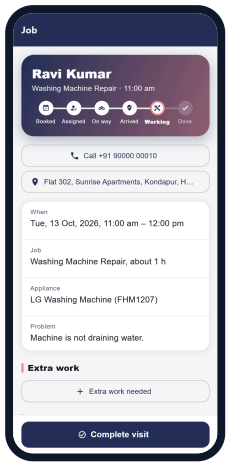
# Phone — open the HomeFix app (or Expo Go), tap your PC's server, log in.
```

1 What HomeFix is and how the pieces fit

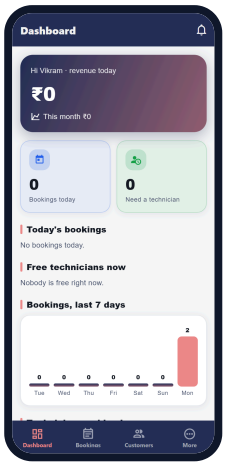
HomeFix is a booking and field-service app for an appliance repair business. **Customers** book a repair and pay, **technicians** run their visits, and the **owner and manager** run the business. All of it lives in one phone app; each person sees only their own area after logging in.



Customer: book a repair



Technician: the job

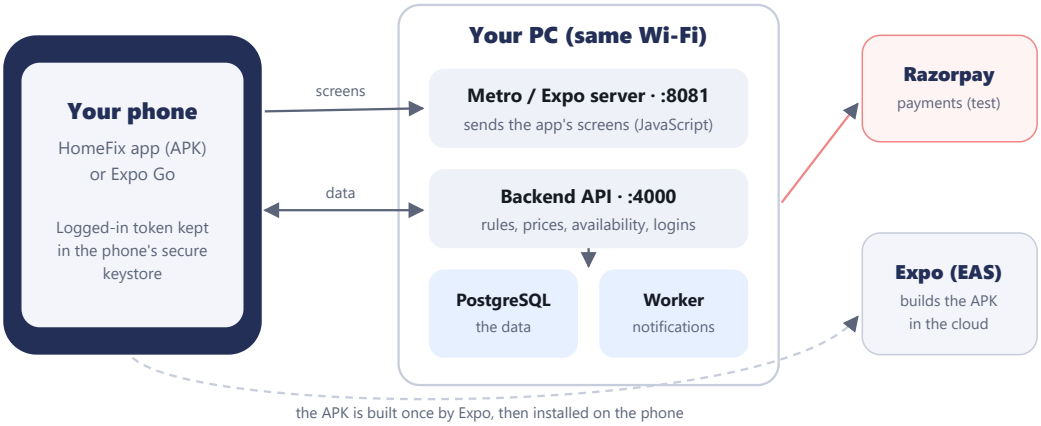


Owner / manager: dashboard

Three projects (three Git repositories)

Repository	What it is	Needed for the phone app?
businesshub-backend	The server (API): Node.js, Express, TypeScript, PostgreSQL via Prisma. Every rule and every amount is decided here.	**Yes** — the app cannot work without it.
businesshub-reactnative	The phone app: React Native with Expo, for customers, technicians and staff. **This guide is about this one.**	Yes — it is the app.
businesshub-frontend	The website (Next.js). Same backend, same rules.	No. Optional.

How they talk to each other



The one idea to remember

The phone only **shows** things and **sends** what the user picked. The backend decides everything: who may do what, which times are free, every price and every allowed status change. That is why the backend must be running for the app to work.

2 What you need before you start

Thing	Why	Where to get it
Windows PC (Mac/Linux also work)	Runs the backend and the app's code server.	—
Node.js 22 LTS or newer	Runs JavaScript/TypeScript tools (npm, npx, Expo).	nodejs.org → LTS installer. Check: `node -v`
Git	Downloads the code.	git-scm.com. Check: `git --version`
PostgreSQL 16 or newer	The database. Remember the password you set for the user `postgres`.	postgresql.org/download/windows
An Android phone	To run the app. Same Wi-Fi as the PC.	—
Expo Go app (optional)	Quick way to run the app without building it. No online payment in Expo Go.	Play Store → "Expo Go"
Expo account (free)	Needed to build the real APK in Expo's cloud.	expo.dev/signup (chapter 7)
Razorpay account (free, optional)	Test keys for "Pay online". No real money.	dashboard.razorpay.com → Test Mode → API Keys
VS Code (optional)	A good editor to read the code.	code.visualstudio.com

Words you will see a lot

Terminal

The window where you type commands. On Windows: press Start, type **PowerShell**, open it. You often need two or three terminals open at once.

npm / npx

`npm install` downloads the project's libraries. `npm run dev` runs a script from `package.json`. `npx tool` runs a tool without installing it.

.env file

A small text file with settings and secrets (passwords, keys). Each project has a `.env.example`; you copy it to `.env` and fill it in. Never shared on GitHub.

IP address

Your PC's address on the Wi-Fi, like `192.168.1.5`. The phone uses it to reach the PC. It can change, so check it each day.

3 Get the code

Put the projects **next to each other in one folder**. The mobile app's tests look for the backend in the folder beside it.

```
# Make a folder for the projects and go into it
mkdir D:\Projects
cd D:\Projects

# Download the backend and the phone app
git clone https://github.com/saierukala/businesshub-backend.git BusinessHub-backend
git clone https://github.com/saierukala/businesshub-reactnative.git businesshub-reactnative
```

Check

You now have `D:\Projects\BusinessHub-backend` and `D:\Projects\businesshub-reactnative`.

4 Start the backend (database + API)

1 Install its libraries

```
cd D:\Projects\BusinessHub-backend
npm install
```

2 Create the settings file

```
copy .env.example .env
notepad .env
```

In Notepad, change these lines and save:

Setting	What to put
`DATABASE_URL`	Replace `YOUR_PASSWORD` with your PostgreSQL password. The database `businesshub` is created for you.
`TEST_DATABASE_URL`	Same password. Used only by the tests (it gets wiped).
`JWT_SECRET`	A long random string. Make one with: `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Optional. Your Razorpay **test** keys (`rzp_test_…`). Empty = "Pay online" is hidden.
`SMTP_URL`	Leave empty. Emails are printed in the terminal instead of sent.

3 Create the database tables and demo data

```
npx prisma migrate dev
npm run seed
```

`migrate` builds the tables, including the rule that makes double booking impossible. `seed` adds demo users, services and a customer (logins in chapter 20).

4 Run the API (keep this terminal open)

```
npm run dev
```

Optional, in another terminal, for in-app notifications: `npm run worker`

Check

Open `http://localhost:4000/health` in a browser. You should see `{"status":"ok", …}`. The full API documentation is at `http://localhost:4000/docs`.

5 Set up the mobile app

1 Install its libraries

```
cd D:\Projects\businesshub-reactnative
npm install
```

At the end you will see `patch-expo-ui: patched ExpoUIModule.kt`. That is expected: a small fix for an Expo bug runs automatically after every install (chapter 15).

2 Find your PC's IP address

```
ipconfig
```

Look under **Wireless LAN adapter Wi-Fi** for **IPv4 Address**, for example `10.166.62.82`.

3 Tell the app where the backend is

```
copy .env.example .env
notepad .env
```

Set the one line to your IP and port 4000, then save:

```
EXPO_PUBLIC_API_URL=http://10.166.62.82:4000
```

Why not "localhost"?

On the phone, "localhost" means the phone itself. The phone must use the PC's Wi-Fi address. If you use the Android emulator instead, use `http://10.0.2.2:4000`.

4 Let the phone reach the PC

- Phone and PC on the **same Wi-Fi**.
- Windows Settings → Network → your Wi-Fi → set it to **Private**.
- The first time Node.js asks "Allow access?", tick **Private networks** and allow.

Check

On the **phone's** browser open `http://<your IP>:4000/health`. It must show `{"status":"ok"…}`. If it doesn't load, nothing else will work yet: see Troubleshooting.

Office or college Wi-Fi?

Many of these block devices from talking to each other. Turn on your phone's **hotspot**, connect the PC to it, run `ipconfig` again and update `.env` with the new IP.

6 Run it: browser, Expo Go, QR codes

There are three ways to run the app. Start with the browser or Expo Go; build the real app (chapter 8) when you want online payments.

Way	Good for	Pay online?
Browser preview	Quick look on the PC, the automated tests.	No (hidden)
Expo Go (Play Store app)	Running on your phone in 2 minutes, no build.	No (hidden)
Development build (your own APK)	Everything, including Razorpay. The way to record the demo.	**Yes**

A. Browser preview (PC only)

```
npm run web -- --port 3000
```

Opens the app in a phone-like browser page. Use port 3000: the backend only accepts browsers from the address in its `FRONTEND_URL` setting.

B. Expo Go on your phone

1. Install **Expo Go** from the Play Store.
2. On the PC: `npx expo start`. A big **QR code** appears in the terminal.
3. Open **Expo Go** → **Scan QR code** and point the camera at the terminal. The first load takes about a minute (a progress bar), then the HomeFix login appears.

Two different QR codes — don't mix them up

QR code	Where you see it	What it does	How often
Server QR	The terminal running `npx expo start`	Opens your **code** from the PC inside Expo Go or the HomeFix app	Every time you open the app
Build QR	The end of an EAS build, and its page on expo.dev	**Downloads and installs the APK**	Once per new build

Useful keys in the Expo terminal

`r` reload the app · `m` open the developer menu · `j` debugger · `Ctrl+C` stop the server.

7 Create your Expo account

Expo makes the tools this app is built with. **EAS** (Expo Application Services) is their cloud: it turns the project into a real Android app (an **APK**) on their computers, so you don't need Android Studio.

1. **Sign up**
   Go to **expo.dev/signup**, choose a username, email and password, then confirm the email Expo sends you.

2. **Ignore the welcome page**
   After signing up, Expo shows "Get your project started" with `npx create-expo-app`. **Don't run it** — that makes a new empty app. You already have the project. Close the page.

3. **Log in from the terminal**

   ```
   cd D:\Projects\businesshub-reactnative
   npx eas-cli@latest login
   ```

   It may open your browser: log in there and you'll see **"Authentication successful"**. The terminal then says **Logged in**.

> **"Your username, email, or password was incorrect"**
> This is your **Expo** password (not Gmail, not GitHub, not the app's login). If you signed up with Google or GitHub, set a password with "Forgot password?" on expo.dev, or use the browser login above.

What happens on expo.dev

- The first build creates a **project** called `businesshub-reactnative` under your account and writes its ID into `app.json` (`extra.eas.projectId`). Commit that change.
- Expo creates and keeps the app's **signing key** (keystore). Android only installs signed apps, and every update must be signed with the same key — so let Expo keep it.
- Every build gets a page with live logs, a **QR code** and an **Install** button. Your builds list: expo.dev → your project → **Builds**.
- The free plan builds in a shared queue (a few minutes' wait is normal) and includes a limited number of builds each month — see expo.dev/pricing.

8 Build the Android app (APK) in the cloud

You need a build only the first time, and again when a library with **native code** is added (for example the Razorpay, camera or photo libraries). Normal screen changes don't need a new build: the app gets them from your PC.

1 **Bump the version (every new build)**

In `app.json`, raise `"version"` (1.0.0 → 1.0.1) and `"android" → "versionCode"` (1 → 2). The app shows them on the login screen and in Settings → About, so you always know which build is on the phone.

2 **Start the build**

```
npx eas-cli@latest build --profile development --platform android
```

3 **Answer its questions (first time only)**

It asks	Answer
Need to install eas-cli... Ok to proceed?	`y`
Which account should own this project?	Your personal account → Enter
Create an EAS project for @you/businesshub-reactnative?	`Y`
Generate a new Android Keystore?	`Y` (Expo keeps it safe)
Install and run the build on an emulator?	`n` (you'll use your phone)

What happens while you wait (15–20 minutes)

Stage	What it means
Compressing and uploading	Your project (without `node_modules` and ignored files) goes to Expo, about 1–2 MB.
Computing the project fingerprint	A summary of the native parts, to tell builds apart. Can take a minute; "taking longer than expected" is normal.
In queue	Waiting for a free build machine (free plan).
Install dependencies	`npm install` on Expo's machine; our Expo fix (`scripts/patch-expo-ui.js`) runs here too.
Prebuild	Expo generates the native Android project from `app.json` and the libraries.
Run gradlew	The Android compiler builds and signs the APK. The longest part.
Build finished	You get a link and a QR code to install it.

Don't want to wait in the terminal?

Add `--no-wait` : it uploads, prints the build page link and returns. Watch the build on expo.dev; the page shows **Install** when it's done.

"adb executable doesn't seem to work ... spawn adb ENOENT"

Appears only if you answered yes to "run on an emulator" and Android Studio isn't installed. The build itself **succeeded**; install it on the phone instead.

9 Install the APK on your phone

1. **Open the build link on the phone**
   Scan the **build QR** with the phone camera, or open the link from the terminal / expo.dev build page in the phone's browser. Log in to Expo if asked.
2. **Tap Install (or Download)**
   A file like `application-….apk` downloads.
3. **Open the file**
   From the download notification, or **Files** → **Downloads**.
4. **Allow installs from this source**
   Android asks once: turn on **Allow from this source** for your browser, go back, tap **Install**.
5. **Play Protect warning**
   Apps that don't come from the Play Store get a warning. Tap **More details** → **Install anyway**. This is normal for your own test builds.

Check

A **HomeFix** app appears in your app list, next to Expo Go.

A newer build?

Install it the same way. Android calls it an **update** and keeps your login.

This is a development build

It gets its screens from your PC, so the PC must be on with the backend and `npx expo start` running, on the same Wi-Fi. It is for testing and recording — not for giving to customers. A stand-alone app needs a "preview/production" build and a deployed backend.

10 Open the app and connect it to your PC

1. On the PC: backend running (`npm run dev`) and the code server: `npx expo start` in the mobile folder.
2. Open **HomeFix**. It shows **Development servers**: tap `http://<your IP>:8081` . Not listed? Scan the **server QR** in the terminal, or tap **Enter URL manually**.
3. Wait about a minute for the first load. The terminal prints `Android Bundled …` . The Navy login screen appears.
4. Under the login form: **Version 1.0.0 (build 1) · code ef852c7, 13 Oct, 6:25 pm**. The first part is the installed APK, the second is the code your PC is serving.

11 Test online payments with Razorpay

Customers can pay a **completed** visit online. The server creates the Razorpay order and decides the amount, Razorpay's own secure screen takes the card/UPI/bank details, and the server checks Razorpay's signature before marking the booking paid. In **test mode** no real money moves.

Before you start

- Backend `.env` has `RAZORPAY_KEY_ID=rzp_test_…` and `RAZORPAY_KEY_SECRET`. Live keys (`rzp_live_…`) are refused on purpose.
- You are using the **HomeFix development build** (not Expo Go, not the browser).
- There is a booking that is **Completed** and **not paid**: the technician pressed "Complete visit" but not "Payment received".

Make a booking to pay (with the demo accounts)

1. **Customer** (ravi@example.test) books a repair.
2. **Manager** (manager@homefix.test) opens it → **Assign technician** → Rahul.
3. **Technician** (rahul@homefix.test) opens the job → **I'm on my way** → **I've arrived** → **Start work**, fills "What did you find?" and "What did you do?" → **Complete visit**. Stop here.

Pay

1. Log in as the **Customer** → **Bookings** → the completed booking.
2. Under **Payment**: a yellow **Test mode** note and **Pay ₹… online**. Tap it.
3. In Razorpay: **UPI** → `success@razorpay`, or **Netbanking** → any bank → **Success** on the test bank page.
4. Back in the app: **"Payment successful"**, the booking shows **Paid ₹… by online** and **View receipt**.

Try a failed payment too

Use `failure@razorpay` (or **Failure** on the bank page). You see an error, the booking stays unpaid, and you can pay again.

Check

The receipt has a receipt number (RC-…), the online payment reference (pay_…) and the right total. Staff see the same payment on the website and in the app.

Never

Put live Razorpay keys, real card numbers or the `.env` file in Git, chats or screenshots.

12 Record the phone screen for the demo

Prepare the phone

- **Same look as the other clips:** in the app, Account → **Settings** → **Light**, app colour **Navy**.
- **Hide the developer gear:** the grey round button at the bottom right is Expo's developer tools. Open the developer menu (shake the phone, or press `m` in the Expo terminal) and turn off its floating button option before recording.
- Turn on **Do not disturb** so no notifications appear.
- Quick Settings (swipe down twice) → add **Screen record** with the pencil icon if it's missing.

Record

1. Open the screen you start on, then Quick Settings → **Screen record** → **no audio** → Start.
2. Move slowly: pause a second on each screen so viewers can read it.
3. Stop from the notification. The video is in Gallery / Photos.

> **Send the original, not WhatsApp**
>
> WhatsApp shrinks videos (to about 480 pixels wide). Copy the file with a USB cable, Google Drive or Nearby Share to keep it sharp.

> **Making it match the Playwright clips**
>
> The phone clip is placed in the same phone frame on the same HD background as the generated clips, with the status bar cropped and the end trimmed.

The demo clips in order

#	Clip	Length
1	01 Customer books a repair	46 s
2	02 Manager assigns a technician	16 s
3	03 Technician on the job	21 s
4	04 Customer approves the extra charge	17 s
5	05 Technician completes the visit	22 s
6	05b Customer pays online (real phone)	31 s
7	06 Manager books for a phone-only customer	34 s
8	07 Owner dashboard and reports	14 s
9	08 Roles and permissions	14 s
10	09 Settings: dark mode	16 s

13 The project's files: what goes where

The mobile repository, `businesshub-reactnative`. Folders are in bold; the grey text says what lives there.

app/	Every file here is a SCREEN (Expo Router: the file path is the screen's address)
_layout.tsx	The root: theme, login check, sends each role to its own area
(auth)/	Logged-out screens: login, register, forgot password, "I have a code"
(customer)/	Customer area
(tabs)/	Bottom tabs: home, bookings, book, account
bookings/[id]/	One booking ([id] = the booking's id) and its receipt
(tech)/	Technician area: today's jobs, the job screen, receipt
(staff)/	Owner / manager area: dashboard, bookings, customers, technicians, reports, reviews, users, services, audit log, roles, settings
src/	Everything that is not a screen file
features/	The real screen content, grouped by topic
booking/	Booking wizard, slot picker, booking detail, payment, receipt, Pay online
staff/	Dashboard, customers, technicians, reports, users, roles…
tech.tsx	The technician's day and job screen
Settings.tsx	Light/dark mode, app colour, About (version)
lib/	Logic without UI
api.ts	The ONE place that calls the backend (adds the login token)
auth.tsx	Who is logged in; login and logout
queries/	One file per topic: "get bookings", "assign technician"… (TanStack Query)
schemas/	Form rules with Zod (e.g. phone must be 10 digits)
format.ts	Dates, times (always India time), money
appearance.tsx	Saves light/dark and app colour on the phone
razorpay.native.ts	Pay online on a real phone; razorpay.ts = the browser stand-in
version.ts	The version line on the login screen and in Settings
ui/	Reusable building blocks
theme.ts	ALL colours: the 8 app colours, light and dark, status colours
Parts.tsx	Card, Hero, StatCard, StatusChip, EmptyState, lists, dialogs
Form.tsx	Text fields, choice lists, buttons
nav.tsx	Header, bottom tab bar, notification bell
e2e/	Playwright end-to-end tests (one per path), run on phone-sized browsers
demo/	Video recorder (npm run demo) and screenshot maker (npm run shots)
scripts/	patch-expo-ui.js: the automatic fix for an Expo crash
docs/	This guide, the LinkedIn video script
assets/	App icon and splash images
app.json	App name, version, icon, permissions, plugins
app.config.js	Adds the git commit and build time to app.json
eas.json	How Expo builds the APK (the "development" profile)
package.json	Libraries and the npm run … scripts
.env	Your backend address (never in Git)

The backend, in one look

src/routes/	URLs (e.g. POST /bookings) and their input checks
src/controllers/	Read the request, call a service, send the answer
src/services/	ALL business rules (booking, availability, payments, visits)
src/db/	Database connection (Prisma)
src/jobs/	Background work: notifications, reminders (npm run worker)
prisma/	schema.prisma (the tables), migrations, seed.ts (demo data)

14 How one screen works, end to end

Example: the customer taps **Confirm booking**.



The screen then refreshes the lists (TanStack Query) and opens the new booking.

Layer	Its only job
Screen (`src/features`)	Show things, collect what the user picks, show loading and errors.
Query hook (`src/lib/queries`)	Name the action ("create a booking"), cache results, refresh lists afterwards.
`src/lib/api.ts`	Send the request with the login token; turn errors into readable messages; a "401" logs you out.
Backend service	Check permissions and every rule, use India time, write history and the audit log.
Database	Store it. A database rule makes two bookings for the same technician at the same time impossible.

Adding a new screen, in short

1) a query in `src/lib/queries`, 2) the content in `src/features`, 3) a small route file in `app/` that renders it, 4) its title in that area's `_layout.tsx`, 5) colours only from `useAppTheme()`.

15 Important things to know

The backend decides

Free times, prices, totals and who may do what are decided by the server. The app never works these out itself. Hiding a button is only for looks; the server checks again.

India time, always

All dates and times are shown in India time (IST), whatever the phone's setting. The server stores times in UTC.

Logging in on the phone

The app logs in with the header `X-Client: mobile`, gets a token, keeps it in the phone's secure keystore and sends it as `Authorization: Bearer …`.

Expo Go vs your own build

Expo Go can't contain Razorpay's native code, so Pay online is hidden there. Your development build has it.

When you need a new APK

Only after adding or upgrading a library with native code, or changing `app.json` plugins/permissions. Screen and logic changes reach the phone from `npx expo start`.

Always bump the version

Before each build, raise `version` and `versionCode` in `app.json`. The login screen shows them, so you know which build you're testing.

The Expo crash fix

`scripts/patch-expo-ui.js` runs after every `npm install`. It stops a crash in Expo SDK 57 (`UnsatisfiedLinkError … clearAllContentOrigins`) when the development build opens the app. Remove it once Expo fixes the bug.

Secrets

`.env` files are ignored by Git. Never commit keys or passwords. Razorpay runs in test mode only.

Your IP changes

New day, new Wi-Fi or a hotspot: run `ipconfig` and update `EXPO_PUBLIC_API_URL`, then restart `npx expo start`.

App colours

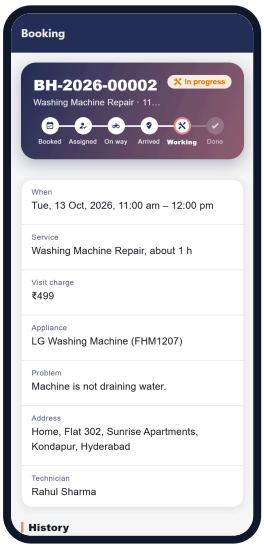
8 palettes (Navy default) and light/dark live in `src/ui/theme.ts`. Screens use `useAppTheme()`; no colour is hard-coded in a screen.

Ports used

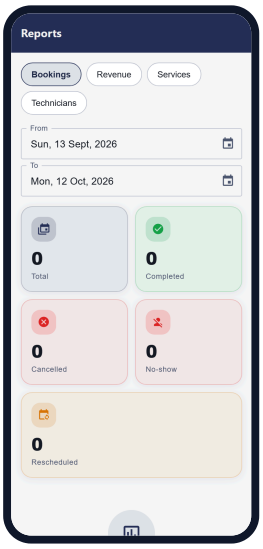
Port	What
4000	Backend API (dev)
8081	Metro / Expo code server
3000	Browser preview of the app (and the website)
4200 / 8200	The mobile E2E tests' own API and app (their own database)

16 Tests, demo videos and screenshots

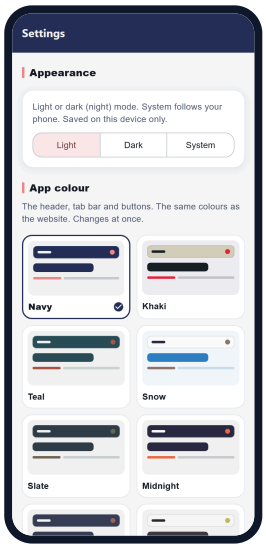
Command	What it checks / makes	Notes
`npm run typecheck`	TypeScript: no type errors anywhere.	Seconds.
`npm test`	Unit tests (India-time dates, money, helpers).	Seconds.
`npx playwright test`	End-to-end on Pixel 7 and iPhone 14 sizes: customer books and cancels, staff books for a customer, technician completes and records cash, owner opens Roles, dark mode persists.	Needs PostgreSQL. Starts its own API and app, wipes and seeds its own database (`businesshub_e2e_mobile`). About 1–2 min.
`npm run demo`	Records the 9 demo clips (Full HD, phone frame) into `demo-videos/`.	About 4–5 min. `DEMO_SCENES=2,5` re-records only those.
`npm run shots`	Screenshots of the main screens in all 8 colours, light and dark, plus a cut-off check.	`SHOTS_PALETTES=navy,teal` , `SHOTS_WIDTH=375` .
`npx expo-doctor`	Checks the Expo setup and library versions.	Should say all checks passed.
`npm run lint`	Code style (ESLint).	



Booking with the visit tracker



Reports



Settings: app colours

17 Command cheat sheet

I want to…	Run (in the right folder)
Start the backend	`npm run dev` (backend)
Start notifications	`npm run worker` (backend)
Reset demo data	`npm run seed` (backend)
Serve the app to the phone	`npx expo start` (mobile) · add `--clear` if things look stale
Open the app in a browser	`npm run web -- --port 3000` (mobile)
Log in to Expo	`npx eas-cli@latest login`

Build the APK	`npx eas-cli@latest build --profile development --platform android`
See my builds	`npx eas-cli@latest build:list` or expo.dev → Builds
Find my PC's IP	`ipconfig`

18 Troubleshooting

You see	Why	Fix
"Can't reach the server…" in the app	Backend not running, wrong IP, different Wi-Fi, or firewall.	Check `http://<IP>:4000/health` on the phone's browser. Run `npm run dev`, update `.env` IP, set Wi-Fi to Private, allow Node.js, or use the phone hotspot.
HomeFix app can't connect to the development server	Port 8081 blocked or `npx expo start` not running.	Start it, allow Node.js in the firewall, or type the URL manually: `http://<IP>:8081`.
App crashes right after opening, `UnsatisfiedLinkError … clearAllContentOrigins`	The Expo SDK 57 bug.	Make sure `npm install` printed "patch-expo-ui: patched", then build a new APK.
No "Pay online" button	Expo Go or browser; no Razorpay keys in backend `.env`; booking not Completed or already paid.	Use the HomeFix build, add test keys and restart the backend, complete a visit without recording payment.
EAS: "username, email, or password was incorrect"	Not your Expo password, or a Google/GitHub sign-up.	Reset it on expo.dev or use `npx eas-cli@latest login` (browser login).
"adb … ENOENT" after a build	You chose to run on an emulator without Android Studio.	Ignore it; install the APK on your phone.
Play Protect blocks the install	App not from the Play Store.	More details → Install anyway.
Old screens / changes don't appear	Cached code.	Press `r` in the Expo terminal, or restart with `npx expo start --clear`.
Browser preview: requests fail (CORS)	The backend only allows its `FRONTEND_URL` address.	Use port 3000, or set `FRONTEND_URL` to the preview's address.
"Port 3000/8081 is in use"	Another app or an old server is running.	Close it, or pick another port (e.g. `--port 8082`).
`prisma migrate` fails to connect	PostgreSQL not running or wrong password.	Start the PostgreSQL service; fix `DATABASE_URL` in the backend `.env`.
Version line shows the old build	The new APK isn't installed yet.	Install the new APK (it's an update), then reopen the app.

19 Glossary

Word	Meaning
React Native	Write the app once in JavaScript/TypeScript and React; it runs as a real Android/iPhone app.
Expo	Tools on top of React Native: ready-made phone features, the code server, the cloud builds.
Expo Go	A ready app from the Play Store that runs Expo projects without building them. Can't hold extra native code.
Development build	Your own version of "Expo Go" with this app's native code inside (Razorpay, photos). Loads screens from your PC.
EAS	Expo Application Services: the cloud that builds (and can publish) the app.
APK	The Android app file you install.
Metro	The code server started by `npx expo start`; it bundles the JavaScript and sends it to the phone.
Native code	Java/Kotlin (Android) or Swift (iOS) inside a library. Needs a new APK when it changes.
Expo Router	Screens are files in `app/`; the folder path is the screen's address.
TanStack Query	Fetches data from the API, caches it and refreshes it.
Zod	Checks form input (and on the server, every request).
JWT / token	The proof you're logged in, sent with every request.
Keystore	The secret key that signs the APK. Expo keeps it for you.
Prisma	The backend's tool to talk to PostgreSQL and to change tables (migrations).
Playwright	Drives a browser automatically: the tests and the demo videos.
Razorpay test mode	Payments that behave like real ones, but no money moves.

20 Demo logins

From the backend's demo data (`npm run seed`). Password for all: `Password@123`. In development the login screen has buttons that fill these in.

Role	Login	Sees
Owner	`owner@homefix.test`	Everything, including users, services, audit log, roles
Manager	`manager@homefix.test`	Bookings for anyone, technicians, reports
Technician	`rahul@homefix.test` (also arjun@, suresh@)	Only their own jobs
Customer	`ravi@example.test`	Only his own bookings, payments, reviews

Demo only

These are shared demo accounts with a well-known password. Never use them on a real, public system.

HomeFix Mobile · Setup, Build & Run Guide · generated from `docs/guide/guide.html` with `node docs/guide/build-guide.cjs`